

LAB2 关系模式设计初稿与规范化分析

成员：俞楚凡、赵敬彦

2026-04-01

1 关系模式初稿 (Relational Schema)

符号说明：下划线 字段 表示主键 (PK)；后缀 (FK) 表示外键。

1.1 人员与权限模块

- 人员基表 (People):
(people_id, name, gender, phone, email)
- 学生表 (Student):
(people_id(FK), student_no, grade, major, dep_id(FK))
- 教师表 (Teacher):
(people_id(FK), staff_no, title, dept_id(FK))
- 用户表 (User):
(user_id, people_id(FK), verification_status, username, password_hash, role_type, dep_id(FK))

1.2 空间地理模块

- 校区表 (Campus):
(campus_id, campus_name, address)
- 建筑表 (Building):
(building_id, building_name, campus_id(FK), building_type, description)
- 地点表 (Location):
(location_id, location_name, building_id(FK), facility_type, description, open_time)

1.3 组织、课程与活动模块

- 院系表 (Department):
(dep_id, dep_name, contact_info, office_location_id(FK), manager_id(FK), description)
- 课程表 (Course):
(course_id, course_name, teacher_id(FK), dep_id(FK), description)
- 校园活动表 (Event):
(event_id, event_name, event_type, start_time, location_id(FK), host_dep_id(FK), description)

1.4 系统日志模块

- 问答记录表 (QueryRecord):
(record_id, user_id(FK), raw_question, query_time, query_result)

2 关系设计

以下四个分块与第 1 节各实体子节一一对应。

2.1 人员与权限模块

- **人员 — 学生** (属于) 1:1
学生是人员的子类型，采用类表继承模式，通过共享主键 *people_id* 关联。每位学生恰好对应一条人员基表记录，反之不一定成立。
- **人员 — 教师** (属于) 1:1
教师是人员的子类型，同样通过 *people_id* 继承人员基表。同一 *people_id* 不应同时出现在 Student 和 Teacher 中。
- **人员 — 用户** (注册) 1:1
一个人员至多注册一个系统用户账号，*User.people_id* $\rightarrow$ *People.people_id*。用户表额外承载身份验证与角色权限信息。

2.2 空间地理模块

- **校区 — 建筑** (拥有) 1:N
一个校区包含多栋建筑，*Building.campus_id* $\rightarrow$ *Campus.campus_id*。删除校区需级联处理其下所有建筑。
- **建筑 — 地点** (拥有) 1:N
一栋建筑包含多个地点(教室、食堂、实验室等)，*Location.building_id* $\rightarrow$ *Building.building_id*。地理层次为：校区 $\supset$ 建筑 $\supset$ 地点。

2.3 组织、课程与活动模块

- **院系 — 学生** (拥有 / 隶属) 1:N
一个院系下有多名学生，*Student.dep_id* $\rightarrow$ *Department.dep_id*。每名学生仅隶属于一个院系。
- **院系 — 教师** (拥有 / 隶属) 1:N
一个院系下有多名教师，*Teacher.dept_id* $\rightarrow$ *Department.dep_id*。每名教师仅隶属于一个院系。
- **人员 — 院系** (负责 / 管理) 1:1
每个院系有一名负责人，*Department.manager_id* $\rightarrow$ *People.people_id*。负责人须为本校在册人员。
- **院系 — 地点** (坐落 / 办公地点) N:1
院系的办公地点位于某一地点，*Department.office_location_id* $\rightarrow$ *Location.location_id*。多个院系可共享同一办公地点。
- **院系 — 课程** (开设) 1:N
一个院系可开设多门课程，*Course.dep_id* $\rightarrow$ *Department.dep_id*。每门课程归属于一个院系。
- **教师 — 课程** (教授) 1:N
一名教师可教授多门课程，*Course.teacher_id* $\rightarrow$ *Teacher.people_id*。每门课程有且仅有一名主讲教师。
- **院系 — 校园活动** (举办) 1:N
一个院系可举办多个校园活动，*Event.host_dep_id* $\rightarrow$ *Department.dep_id*。

- **地点 — 校园活动**（位于 / 举办地点） 1:N
一个地点可承办多个校园活动， $Event.location_id \rightarrow Location.location_id$ 。活动时间不重叠时同一地点可复用。

2.4 系统日志模块

- **用户 — 问答记录**（产生） 1:N
一个用户可产生多条问答记录， $QueryRecord.user_id \rightarrow User.user_id$ 。记录仅追加写入，用于审计与问答质量分析。

3 主键与外键设计说明

目标表	主键 (PK)	外键 (FK)	设计依据
Student/Teacher	<i>people_id</i>	<i>people_id</i> $\rightarrow$ <i>People</i>	类表继承模式，解耦基础信息与角色属性。
Building	<i>building_id</i>	<i>campus_id</i> $\rightarrow$ <i>Campus</i>	建立地理包含关系，支持按校区统计。
Location	<i>location_id</i>	<i>building_id</i> $\rightarrow$ <i>Building</i>	细化建筑内部空间，区分设施类型。
Event	<i>event_id</i>	<i>location_id</i> $\rightarrow$ <i>Location</i>	关联具体地点，支持查询实时动态。

4 规范化分析 (Normalization Analysis)

4.1 数据冗余 (Data Redundancy)

本设计严格遵循 **3NF**（第三范式）。通过在 *Course* 表中仅存储 *teacher_id* 而非教师姓名，以及在 *Location* 表中通过 *building_id* 级联，极大地降低了数据空间冗余，保证了信息的单一事实来源。

4.2 更新异常 (Update Anomalies)

由于建立了完整的外键约束，若“校区名称”发生变更，只需在 *Campus* 表中修改一次。由于 *Building* 和 *Location* 的层级关系是通过外键维护的，不会出现部分数据未更新导致的空间层级混乱。

4.3 查询支持情况

- **复杂查询**：支持多表连接操作。例如查询某校区的所有食堂：

$$Result = Campus \bowtie Building \bowtie \sigma_{type='食堂'}(Location)$$

- **统计功能**：可直接通过 *Course* 表的 *dept_id* 聚合统计各院系课程数量。
- **NL2SQL 基础**：清晰的实体边界和外键关联为自然语言转 SQL 的语义解析提供了稳定的元数据基础。